

Software Engineer

Product Engineer

Frontend Developer

가천대학교 컴퓨터공학과 GPA 3.99 / 4.5

PROFILE

React·TypeScript로 모바일 앱·브라우저 확장에서 함께 쓰는 WebView 제품과 데이터 분석 UI를 개발해왔습니다. 공통 화면과 플랫폼별 기능의 역할을 나누고, BFF로 외부 서비스 연동을 정리했습니다. 렌더링 성능 개선과 모노레포 전환을 맡았으며, 프론트엔드 파트 리더로 업무 배분과 우선순위 결정도 담당했습니다.

EXPERIENCE

2024.09 - 현재

Frontend Developer

2년

파이레프테크놀로지

Frontend Developer

WebView 거래 제품과 BFF, 블록 스캐너 성능 개선, 사내 AI Agent 런타임 개발을 맡고 있습니다.

TypeScript

React

Next.js

TailwindCSS

TanStack Query

Jotai

NestJS

wagmi · viem

Chakra UI

Vitest

Playwright

Sentry

- BiFi·Swap·BTCFi 거래 화면을 모바일 앱·Chrome Extension에서 함께 사용하도록 공통 WebView와 postMessage 연동 구조 구현
- 파트너사 iOS·Android 지갑에 BTCFi WebView를 연동하고 일반 웹과 거래 로직을 공유하도록 구성
- 블록 스캐너 주요 목록에 단계적 렌더링과 WebSocket 배치를 적용해 로컬 Lighthouse 기준 LCP 평균 **45%**, TBT 평균 **74%** 단축
- Swap의 견적·거래 데이터 생성·상태 조회를 NestJS BFF로 분리해 화면에서 외부 SDK 연동과 오류 처리를 직접 다루던 구조 정리

2021.07 - 2024.09

3년 2개월

Full-stack Engineer

→ Frontend Developer

→ Frontend Part Leader

스마트마인드

Frontend Developer → Frontend Part Leader

AI 데이터 분석 플랫폼 Workspace의 SQL Editor·Query Viewer 모듈을 개발했으며, pnpm·Turborepo 모노레포 전환과 Microfrontend 설계를 주도했습니다. 후반에는 프론트엔드 파트 리더로 팀 운영을 담당했습니다.

TypeScript

React

Next.js

Vite

Monaco Editor

ANTLR

pnpm · Turborepo

Playwright

FastAPI

Docker Compose

- pnpm·Turborepo 모노레포와 빌드 캐시를 도입해 당시 측정 기준 앱별 빌드 시간을 **7분 → 1분**으로 단축
- 번들 분리·dynamic import로 Workspace 초기 로드 범위를 줄여 당시 측정 기준 First Load 최대 **40%** 단축
- AI 팀이 정의한 ANTLR 문법을 Monaco Editor에 연결해 SQL 구문 강조·오류 진단·키워드 자동완성 구현
- Microfrontend 설계·구현 — iframe+postMessage(Lab·Main 간 통신), module federation(Query Manager·File Manager)
- 비정형 데이터 Query Viewer 성능 개선 — 서버 페이지네이션·가상 스크롤·미디어 lazy loading으로 대용량 결과 렌더링 안정화
- 2023년부터 본인 포함 2~3명 프론트엔드 파트의 스크럼 운영·업무 배분·우선순위 결정 담당

01 /

모바일 WebView 공통화·파트너 연동

- **Pockie miniDApp** — BiFi·Swap·BTCFi 화면을 공통 WebView로 배포하고, 지갑 서명·전송은 모바일 앱·Chrome Extension에서 처리하도록 역할 분리
- **Pockie miniDApp** — iOS·Android·Extension과 앱 버전별 feature flag를 BFF config v2에 구현해 클라이언트 재배포 없이 기능 노출 제어
- **BTCFi Partners** — 파트너사 iOS·Android 지갑에 WebView 거래 화면을 연동하고, postMessage로 앱과 통신하며 일반 웹과 거래 로직 공유

02 /

블록 스캐너(Explorer) 성능 최적화

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s 1.9s	1,010ms 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s 2.2s	1,330ms 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s 1.8s	510ms 190ms	LCP ↓45% · TBT ↓63%

- 최대 75개 row를 노출하는 테이블에서 가상화 적용 시 레이아웃 공백 문제가 있어 롤백하고, 초기 row 수를 제한한 단계적 렌더링으로 전환
- WebSocket **30초 배치 처리**로 실시간 업데이트에 따른 반복 렌더링 축소

03 /

DApp 데이터 조회 구조 최적화

- **Gas Top-up** — SDK의 중복 자산 조회와 불필요한 vault 조회를 제거하고 갱신 주기를 조정해 최초 API 호출을 **25~46회→17회**, 주기 호출 분당 **25회→1~19회**로 축소
- **BiFi WebView** — bridge pair 최초 1회+pair별 N회 조회를 outbound·inbound 각 1회로 재구성하고 React Query 캐시를 적용해 반복 조회 제거
- **BiFi WebView** — balance 조회를 background dispatcher 경유에서 클라이언트 단일 호출로 바꾸고 거래 성공 직후 즉시 갱신

04 /

사내 AI Agent 런타임 개선

- 요청별 프로세스 실행을 Codex App Server 세션 기반으로 전환하고, 작업에 필요한 MCP·권한만 허용하도록 실행 정책 구성
- 자동 코드 리뷰에 confidence·CI 상태 검증을 추가하고 최종 merge는 사람이 확인하도록 운영

CERTIFICATIONS & ACTIVITIES

AWS Certified Solutions Architect – Associate (2026)
정보처리기사 (2019)

OSSCA — githru-vscode-ext (2023)
SmileGate Membership AI 1기 (2021)
OSSCA — TS Handbook 한글화 (2020)
AUSG (AWSKRUG University Student Group) 1기 (2019)

LANGUAGES

영어 — 기초 비즈니스 (Opic IM1)
일본어 — 기초 비즈니스 (JLPT N3)